Linguaggi

UniVR - Dipartimento di Informatica

Fabio Irimie

2° Semestre 2025/2026

Indice

1 Linguaggi di programmazione **2**
1.1 Obiettivo dei linguaggi di programmazione 2
1.2 Aspetti di progettazione 3
1.2.1 Tipi di linguaggi 4
1.3 Implementare un linguaggio di programmazione 5

2 Esecuzione dei linguaggi **5**
2.1 Notazione 6
2.2 Interprete 6
2.2.1 Struttura dell'interprete 6
2.3 Compilatore 7
2.3.1 Struttura del compilatore 8
2.3.2 Prima proiezione di Futamura 9

3 Descrivere i linguaggi di programmazione **10**
3.1 Sintassi 10
3.1.1 Strumenti di definizione della sintassi 11
3.1.2 Definizione non formale 12
3.2 Analisi semantica 14
3.2.1 Semantica 14
3.2.2 Induzione strutturale 14

1 Linguaggi di programmazione

Un linguaggio interpretato è un linguaggio di programmazione in cui le istruzioni vengono eseguite direttamente, senza la necessità di essere compilate in un formato eseguibile. Un interprete è rappresentato da un modello di esecuzione, cioè un **control flow graph** (CFG) che rappresenta il flusso di controllo del programma.

I programmi si possono interpretare in diversi modi:

- **Interpretazione diretta**: l'interprete esegue direttamente le istruzioni del programma e segue il flusso di controllo rappresentato dal CFG restituendo i risultati.
- **Interpretazione del segno**: l'interprete esegue le istruzioni del programma ma invece di restituire i risultati, restituisce un segno che rappresenta il risultato. Questo è utile quando non si riesce a restituire un risultato concreto, ad esempio quando un algoritmo non termina.
- **Analisi statica**: l'interprete analizza il programma senza eseguirlo, ad esempio per verificare la correttezza del codice o per ottimizzare le prestazioni.

1.1 Obiettivo dei linguaggi di programmazione

L'obiettivo è creare un modello formale che riesca ad eseguire i costrutti necessari per eseguire un programma. I componenti principali sono:

- **Dati**
- **Algoritmi**: Calcolo da eseguire sui dati, cioè calcola funzioni.

Definizione 1.1 (Algoritmo)**.** Un algoritmo è una sequenza di passi primitivi finita. L'algoritmo è descritto attraverso un programma in un linguaggio di programmazione.

Definizione 1.2 (Linguaggio di programmazione)**.** Un linguaggio di programmazione è uno strumento formale per descrivere algoritmi applicati ai dati.

Definizione 1.3 (Programma)**.** Il programma è la realizzazione nel linguaggio di programmazione di un algoritmo (concetto astratto). È una rappresentazione sintattica, cioè rispetta delle regole sintattiche del linguaggio di programmazione, finita di comportamenti potenzialmente infiniti. I comportamenti potenzialmente infiniti sono funzioni calcolate dall'algoritmo rappresentato (o implementato) dal programma:

$$f : \mathrm{Dom} \to \mathrm{Codom} \cup \{\uparrow\}$$

Il linguaggio di programmazione deve considerare strumenti per descrivere **metodi di calcolo**

Alcuni esempi di linguaggi sono:

- **Linguaggio matematico**: notazione rigorosa per rappresentare funzioni. Questo linguaggio non è adatto per descrivere algoritmi perchè non ha strumenti che descrivono **come** calcolare funzioni.
  - Paradigma funzionale: nascono dal linguaggio matematico per adattarlo alla descrizione di algoritmi. Derivano dal lambda calcolo e alcune implementazioni sono Haskell o Lisp.
- **Linguaggi logici**: notazione rigorosa con regole-assiomi che permettono di specificare un processo di calcolo attraverso regole di derivazione. Il calcolo è rappresentato da un processo di dimostrazione. Manca la descrizione di calcoli potenzialmente infiniti.
  - Paradigma logico: nascono dal linguaggio logico per adattarlo alla descrizione di algoritmi. Derivano dal calcolo dei predicati e un esempio di implementazione è Prolog.
- **Linguaggi di programmazione**: notazione rigorosa, tramite grammatiche CF, con:
  - **Regole**: descrivono la semantica delle istruzioni
  - **Istruzioni**: permettono di descrivere i passi di calcolo

Esempio 1.1. Alcuni esempi di applicazioni dei linguaggi di programmazione sono:

- Applicazioni scientifiche: Fortran, ecc...
- Applicazioni economiche: Cobol, ecc...
- Applicazioni AI: Lisp, ecc...
- Applicazioni sistemi: C, ecc...
- Applicazioni web: JavaScript, Java, ecc...

1.2 Aspetti di progettazione

Gli aspetti per scegliere un linguaggio di programmazione sono:

- **Readability** (Leggibilità): quanto è facile leggere e comprendere i programmi, cioè:
  - Sintassi chiara
  - Nessuna ambiguità
  - Facilità di analisi del codice

- **Writability** (Facilità di scrittura): quanto è facile o agevole scrivere programmi.
- **Reliability** (Affidabilità): quanto il linguaggio è conforme alle specifiche e quanto riduce l'introduzione di errori di programmazione.
- **Cost** (Costo):
  - Costi di efficienza
  - Costi di apprendimento
  - Costi di utilizzo in modo corretto

1.2.1 Tipi di linguaggi

I linguaggi di programmazione si possono classificare come:

- **Linguaggi di basso livello**: sono linguaggi vicini alla macchina fisica, come ad esempio l'assembly o il linguaggio macchina.
- **Linguaggi di alto livello**: sono linguaggi più vicini al linguaggio umano, come ad esempio:
  - Linguaggi funzionali
  - Linguaggi imperativi
  - Linguaggi logici

  I linguaggi si possono ulteriormente dividere in:

  - **Linguaggi puri**: I linguaggi imperativi e alcuni linguaggi funzionali sono detti **funzionali impuri** perchè permettono di descrivere algoritmi con variabili e assegnamenti che possono ricevere degli "effetti collaterali" (side effects) cioè modifiche dello stato del programma. Quindi la variabile è un'astrazione della cella di memoria, di conseguenza lo stesso nome in punti diversi può avere valori diversi.
  - **Linguaggi puri**: I linguaggi logici e alcuni linguaggi funzionali sono detti **funzionali puri** perchè le variabili sono matematiche e quindi **immutabili**, cioè non possono essere modificate dopo essere state assegnate. In questo modo si evitano gli effetti collaterali.

  I linguaggi di alto livello si possono dividere in base al paradigma di programmazione:

  - **Object Oriented**: linguaggi che permettono di strutturare il codice in oggetti con attributi e metodi.
  - **Procedurali**: linguaggi che permettono di strutturare il codice in procedure o funzioni.
  - **Strutturati**: linguaggi che permettono di strutturare il codice in blocchi di istruzioni con costrutti di controllo come if, while, for, ecc...
  - **Dichiarativi**: linguaggi che si basano sulla descrizione del problema piuttosto che sulla descrizione del processo di calcolo.

1.3 Implementare un linguaggio di programmazione

Le caratteristiche che hanno effetto sulla progettazione di un linguaggio di programmazione sono:

- L'architettura della macchina su cui è implementato
- Le metodologie di progettazione di programmi, cioè i paradigmi di programmazione

2 Esecuzione dei linguaggi

L'obiettivo dei linguaggi è **eseguire i programmi**. Per poter eseguire un programma serve un altro programma che lo esegue che è detto **macchina astratta** per il linguaggio di programmazione.

Definizione 2.1 (Macchina astratta)**.** Dato un linguaggio di programmazione L, la macchina astratta per L M_L è un insieme di **strutture dati** e **algoritmi** che permettono di memoriazzare e eseguire programmi scritti in L.

Intuitivamente è la macchina il cui linguaggio è esattamente L.

- Per ogni macchina astratta M_L esiste un unico linguaggio L che è il linguaggio riconosciuto ed eseguito da M_L
- Per ogni linguaggio di programmazione L possono esistere infinite macchine astratte per L

Esempio 2.1. La macchina fisica è la **macchina astratta** per il linguaggio macchina. Mentre tutte le macchine astratte per qualsiasi altro linguaggio di programmazione di alto livello sono software, quindi altri programmi che traducono le istruzioni del linguaggio di programmazione in un'istruzione in un livello più basso.

Definizione 2.2 (Linguaggio macchina)**.** Data una macchina M, il linguaggio macchina di M L_M è il linguaggio che ha come stringhe quelle direttamente interpretabili da M.

Se si ha una pila di macchine astratte $\{M_{i-1}, M_i, M_{i+1}, M_{i+2}\}$ la macchina M_i ha il suo linguaggio macchina L_i che utilizza il linguaggio macchina L_{i-1} della macchina M_{i-1} per eseguire i programmi scritti in L_i. Inoltre il linguaggio L_i può essere usato ai livelli superiori.

Implementare L equivale a realizzare la macchina astratta M_L. Per realizzare M_L ci sono due modi:

1. **Compilazione**: Tradurre L in un linguaggio macchina sottostante

2. **Interpretazione**: Eseguire L sulla macchina sottostante

Definizione 2.3 (Compilazione)**.** La compilazione è una traduzione esplicita di un linguaggio L in un linguaggio L'.

Definizione 2.4 (Interpretazione)**.** L'interpretazione è una traduzione implicita di un linguaggio L che viene **simulato** in L'.

2.1 Notazione

La notazione che verrà utilizzata per descrivere i programmi è la seguente:

- Prog^L: Insieme dei programmi scritti in L
- D: Insieme dei dati manipolati dai programmi in L
- $P^L \in \mathrm{Prog}^L \quad P^L : D \to D$:

$$\forall \mathrm{input} \in D.\ P^L(\mathrm{input}) = \mathrm{output} \in D$$

  cioè il programma P^L è una funzione che prende in input un dato e restituisce un dato in output.

2.2 Interprete

L'interprete è la realizzazione della macchina astratta con soluzione interpretativa (2).

Definizione 2.5 (Interprete)**.** Un interprete per il linguaggio L (scritto nel linguaggio L_0) è un programma $I^{L_0 L} : (\mathrm{Prog}^L \times D) \to D$ che prende in input un programma P^L e un dato e restituisce in output il risultato dell'esecuzione di P^L su input. Questa è la realizzazione della **macchina di Turing universale**.

$$\underbrace{I^{L_0 L}(P^L, \mathrm{input}) = \mathrm{output} \quad \text{t.c.} \quad P^L(\mathrm{input}) = \mathrm{output}}$$

$$I^{L_0 L}(P^L, \mathrm{input}) = P^L(\mathrm{input}) \Rightarrow \text{Macchina universale}$$

2.2.1 Struttura dell'interprete

La struttura dell'interprete è data dal seguente pseudocodice:

```
begin
  go = true;
  while go do
    fetch(opcode, opinfo) at PC; // Program counter
    decode(opcode, opinfo)

    if opcode needs arguments then
```

```
      fetch(args)

    case opcode
      opcode1: execute(opcode1, args)
      opcode2: execute(opcode2, args)
      ...
      halt: go = false

    if opcode has result then store(result)

    PC = PC + sizeof(opcode)

end
```

Esempio 2.2. Un semplice autointerprete, cioè un interprete scritto nel linguaggio che interpreta.

```
input P, d;
pc = 2;
store = [in -> d, out -> 0, x_1 -> 0, x_2 -> 0, ...];
while pc < length(P) do
  # Find the pc-th instruction
  instruction = lookup(P, pc);

  # Syntax rules
  case instruction of
    skip: pc = pc + 1;
    x = e: store = store[x -> eval(e, store)]; pc = pc + 1;
    ...
  endcase
endwhile;

output store[out]


# Semantic rules
eval(e, store) = case e of
  constant: e
  variable: store(e)
  e1 + e2: eval(e1, store) + eval(e2, store)
  e1 - e2: eval(e1, store) - eval(e2, store)
  e1 * e2: eval(e1, store) * eval(e2, store)
  ...
endcase
```

Dove `in -> d` rappresenta l'assegnamento del dato di input alla variabile `in`.

L'interprete **realizza** la macchina astratta M_L di un linguaggio di programmazione L partendo da un linguaggio L_0 con una macchina astratta M_0 direttamente eseguibile passando attraverso una macchina intermedia M_I (interprete).

2.3 Compilatore

Definizione 2.6. Il compilatore da L a L_0 è un programma del tipo:

$$C^{LL_0} : \mathrm{Prog}^L \to \mathrm{Prog}^{L_0}$$

che prende in input un programma P^L e restituisce in output un programma P^{L_0}.

$$\forall P^L \in Prog^L \,.\, C^{LL_0}(P^L) = P^{L_0} \quad \text{t.c.} \quad \forall d \in D \,.\, P^{L_0}(d) = P^L(d)$$

Non ci sono accessi alla memoria o a dati. Verificare che due programmi siano uguali è un problema non decidibile, quindi il compilatore deve essere costruito in modo da garantire che i programmi compilati siano equivalenti a quelli originali.

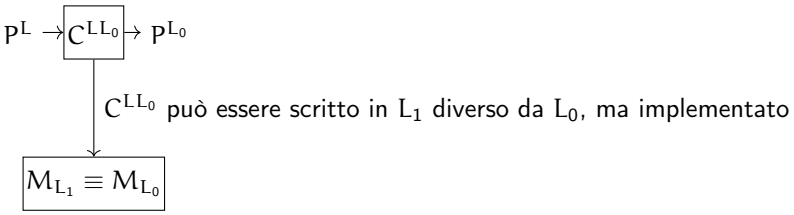

Figura 1: Rappresentazione grafica del compilatore

- Per eseguire il programma bisogna applicare P^{L_0} all'input.
- Il compilatore traduce un codice in un codice diverso, separando la traduzione dall'esecuzione.

2.3.1 Struttura del compilatore

La struttura di un compilatore è data da diversi componenti che lavorano insieme. I componenti principali sono:

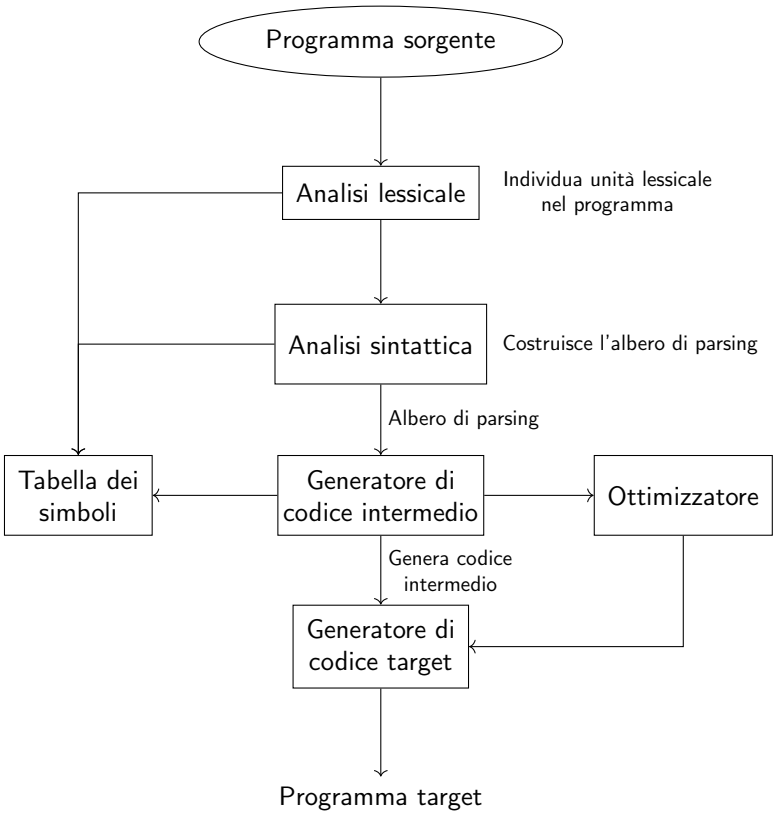

Figura 2: Struttura di un compilatore

Il compilatore **traduce** un programma P^L scritto in un linguaggio di programmazione L in un programma P^{L_0} scritto in un linguaggio L_0 con una macchina astratta M_0 direttamente eseguibile, passando attraverso una macchina intermedia M_I (compilatore).

2.3.2 Prima proiezione di Futamura

Un compilatore può essere ottenuto **specializzando** un interprete su un programma.

Definizione 2.7 (Specializzatore)**.** Uno specializzatore per un linguaggio di programmazione L è un programma che realizza la funzione:

$$\mathrm{Spec}_L : (\mathrm{Prog}^L \times D) \to \mathrm{Prog}^L$$

tale che:

$$\forall P^L \in \mathrm{Prog}^L, \forall d \in D \ .$$
$$\mathrm{Spec}_L(P^L, d) = P^L_d$$
$$\text{t.c. } P^L(d, \mathrm{input}) = P^L_d(\mathrm{input})$$

Quindi d non è più un dato di input, ma è incorporato nel codice.

Lo specializzatore fornisce un legame tra interprete e compilatore. Infatti specializzando un interprete (scritto in L) si ottiene un compilatore da L a L_0.

$$I(P^L, d) \Rightarrow Spec(I, P^L)$$

$Spec$ è un programma scritto in L_0 che integra nel codice l'esecuzione di P^L.

3 Descrivere i linguaggi di programmazione

I linguaggi di programmazione sono dei linguaggi, quindi hanno in caratteristiche in comune con i linguaggi e quindi anche quelli naturali. Le caratteristiche sono:

- **Sintassi**: Regole di formazione. Determinano quali sono le frasi ben formate nel linguaggio e descrive relazioni tra simboli
- **Semantica**: Attribuzione del **significato** ai simboli. Determina il significato delle frasi ben formate nel linguaggio e descrive relazioni tra simboli e significati.
- **Pragmatica**: Regole di buon utilizzo del linguaggio.
- **Implementazione** (Caratteristica aggiunta per i linguaggi di programmazione): Esistono strumenti che eseguono frasi ben formate del linguaggio.

Esempio 3.1. Il numero è soltanto sintassi, mentre il significato è il valore che assume:

$$4 \quad \text{(sintassi)} \rightarrow \text{significato} \quad 4$$

$$IV \quad \text{(sintassi)} \rightarrow \text{significato} \quad 4$$

3.1 Sintassi

Le componenti principali della sintassi sono:

- **Parola**: stringa su un alfabeto Σ
- **Frase**: sequenza di parole ben formata, cioè che rispetta le regole della sintassi
- **Linguaggio**: insieme di frasi ben formate

  Esempio 3.2. Prendiamo ad esempio il seguente linguaggio:

  $$\{a^n b^n \mid n \in \mathbb{N}\}$$

  - **Alfabeto**: $\Sigma = \{a, b\}$
  - **Parole**: $\varepsilon, a, b, aa, ab, ba, bb, aaa, aab, ...$
  - **Linguaggio**: $\{a^n b^n \mid n \in \mathbb{N}\}$

- **Lessema**: una parola (costituita da soli terminali nell'alfabeto) che ha un significato, cioè è una parola che rappresenta un concetto o un oggetto nel linguaggio.

Esempio 3.3. Alcuni esempi di lessemi sono:

- `if exp then com else com`:
  I lessemi sono `if`, `then`, `else`, mentre `exp` e `com` non sono lessemi perchè rappresentano un'espressione e un comando, cioè sono dei simboli non terminali.

- **Token**: categorie sintattiche, cioè simboli non terminali

Esempio 3.4. Alcuni esempi di token sono:

`if exp then com else com`:
I token sono `exp` e `com` perchè sono simboli non terminali.

3.1.1 Strumenti di definizione della sintassi

Gli strumenti per definire la sintassi di un linguaggio di programmazione sono:

- **Riconoscitore**: è uno strumento per il **riconoscimento** di stringhe appartenenti al linguaggio. Ad esempio gli automi a stati finiti, utilizzati nell'analisi lessicale per il riconoscimento dei lessemi.
- **Generatore**: è uno strumento per la **generazione** di stringhe appartenenti al linguaggio. Ad esempio le grammatiche CF, utilizzate nella fase di parsing della compilazione per riconoscere programmi scritti in un linguaggio di programmazione.

Definizione utile 3.1. Un linguaggio di programmazione è dipendente dal contesto, quindi si cerca di svilupparli "a livelli", cioè si fa prima tutto ciò che si può fare con gli automi, poi tutto ciò che si può fare con le grammatiche CF, e così via.

Definizione 3.1. Una grammatica context free è definita da:

$$G = \langle V, T, P, S \rangle$$

dove:

- V: è un insieme finito di simboli non terminali (variabili)
- T: è un insieme finito di simboli terminali

$$V \cap T = \varnothing$$

- P: è un insieme finito di procedure:

$$A \to \alpha \quad A \in V, \alpha \in (V \cup T)^*$$

- $S \in V$: è il simbolo iniziale

In un linguaggio di programmazione la grammatica CF è mappata nel seguente modo:

- V: categorie sintattiche (ad esempio le espressioni, le variabili, ecc...)
- T: vocabolario di parole (`if`, `then`, `else`, ecc...)
- P: regole di formazione
- $S \in V$: categoria della frase (il programma)

Definizione utile 3.2 (Notazione BNF)**.** Per i linguaggi di programmazione viene utilizzata la notazione BNF (Backus-Naur Form) che cambia i simboli utilizzati per rappresentare i componenti della grammatica CF, ad esempio:

$$\rightarrow \quad \text{diventa} \quad ::=$$

Un linguaggio BNF è CF.

Esempio 3.5. Costruiamo una grammatica per espressioni booleane:

$$G = \langle \{\mathrm{Exp}\}, \{0, 1, \mathrm{or}, \mathrm{and}, \mathrm{not}, (,)\}, P, \mathrm{Exp} \rangle$$

$$P = \begin{cases} \mathrm{Exp} \rightarrow 0 \\ \mathrm{Exp} \rightarrow 1 \\ \mathrm{Exp} \rightarrow (\mathrm{Exp}\ \mathrm{or}\ \mathrm{Exp}) \\ \mathrm{Exp} \rightarrow (\mathrm{Exp}\ \mathrm{and}\ \mathrm{Exp}) \\ \mathrm{Exp} \rightarrow (\mathrm{not}\ \mathrm{Exp}) \end{cases}$$

$$= 0|1|(\mathrm{Exp}\ \mathrm{or}\ \mathrm{Exp})|(\mathrm{Exp}\ \mathrm{and}\ \mathrm{Exp})|(\mathrm{not}\ \mathrm{Exp})$$

3.1.2 Definizione non formale

Un linguaggio si può definire anche in un modo non formale, definendo i costrutti base:

- Niente dichiarazioni
- Solo variabili ed espressioni booleane
- Istruzioni:
  - Assegnamento
  - Composizione sequenziale
  - Comando iterativo (ciclo)

- Comando condizionale (if)

Il linguaggio è definito su livelli, partendo dal basso verso l'alto:

1. Notazione della grammatica
2. Linguaggio regolare per riconoscere gli identificatori e le parole riservate al linguaggio di programmazione
3. Espressioni booleane che parte dai simboli definiti ai livelli sottostanti

$$B \rightarrow \mathtt{true} \mid \mathtt{false} \mid (B \text{ or } B) \mid (B \text{ and } B) \mid (\text{not } B) \mid x$$

4. Comandi atomici, ad esempio l'assegnamento. Utilizza gli identificatori, un simbolo di assegnamento e un espressione (booleana in questo caso).

$$A \rightarrow x := B$$

5. Comandi, ad esempio l'if o il ciclo. Utilizzano i comandi atomici e le espressioni booleane.

$$C \rightarrow A \mid C; C \mid \mathtt{while}\ B\ \mathtt{do}\ C \mid \mathtt{if}\ B\ \mathtt{then} C\ \mathtt{else}\ C$$

6. Programmi, cioè sequenze di comandi.

$$P \rightarrow \{C\}$$

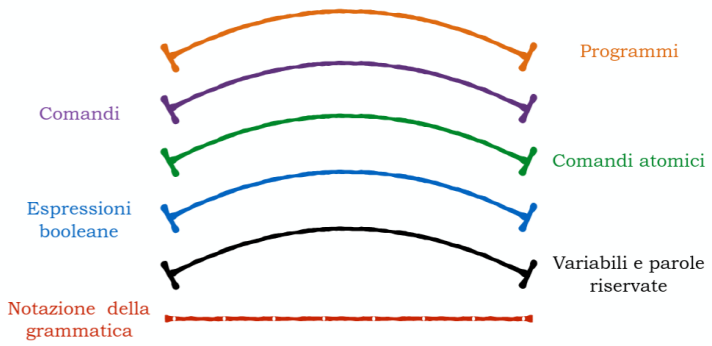

Figura 3: Rappresentazione grafica dei livelli di un linguaggio di programmazione

Si utilizzano espressioni parentesizzate per evitare ambiguità. Se non si utilizzassero le parentesi si potrebbe scegliere di eseguire prima un'operatore invece di un altro e questo porterebbe a risultati diversi dalla stessa espressione.

3.2 Analisi semantica

L'analisi semantica verifica i **vincoli sintattici**, ad esempio che il numero di parametri formali sia uguale al numero di parametri attuali, oppure che l'uso di una variabile sia possibile solo dopo che sia stata dichiarata. Questi sono vincoli sul codice, quindi **non catturati dalla grammatica** e si dicono **contestuali**.

- Sintassi $\rightarrow$ Grammatica CF
- Semantica $\rightarrow$ Tutti i processi di verifica ed esclusione che non sono descritti dalla sintassi

3.2.1 Semantica

La semantica è l'attribuzione di significato ai simboli. Il significato è un'entità autonoma che esiste indipendentemente dai simboli usati per descriverlo. La semantica è più complessa della sintassi perchè deve essere:

- **Esatta**: deve avere una descrizione precisa e non ambigua di ciò che ci si deve aspettare da ogni costrutto sintatticamente corretto per sapere a priori cosa succederà durante l'esecuzione
- **Flessibile**: non deve anticipare scelte che possono essere richieste all'implementazione

3.2.2 Induzione strutturale

L'induzione strutturale permette di dimostrare proprietà di oggetti descritti in modo induttivo o ricorsivo. Una grammatica descrive in modo induttivo il linguaggio (insieme di strunche) che genera, quindi si può utilizzare l'induzione strutturale per dimostrare proprietà delle stringhe generate.

Definizione 3.2. Per dimostrare una proprietà Π su tutti gli elementi di un insieme definito in modo strutturale da una grammatica bisogna:

1. (Caso base): Dimostrare la proprietà su tutti gli elementi base, cioè quelli che sono esplicitamente inseriti nell'insieme di stringhe generate dalla grammatica.
2. (Passo induttivo): Dimostrare la proprietà sugli elementi composti sotto **l'ipotesi induttiva** che la proprietà valga per gli elementi che la compongono

Esempio 3.6. Definiamo l'insieme $X \subseteq \mathbb{N}$:

$$\begin{cases} \text{Base} & : 0 \in X \\ \text{Passo induttivo} & : \text{se } n \in X \text{ allora } n + 3 \in X \end{cases}$$

La proprietà è $X = 3\mathbb{N}$

La proprietà dimostrabile in modo strutturale è: $X \subseteq 3\mathbb{N}$:

1. **Caso base**: 0 è la base di X e $0 \in 3\mathbb{N}$
2. **Passo induttivo**: dimostriamo $\forall n.\ n \in X \Rightarrow n \in 3\mathbb{N}$.
   Prendiamo $n \in X$, l'ipotesi induttiva è la seguente:

   $$n \in X \Rightarrow n \in 3\mathbb{N}$$

   bisogna dimostrare che $n + 3 \in 3\mathbb{N}$. Quindi sappiamo che

   $$\begin{aligned} n \in 3\mathbb{N} &\Rightarrow \exists i.\ n = 3i \\ &\Rightarrow n + 3 = 3i + 3 = 3(i+1) \in 3\mathbb{N} \end{aligned}$$

Esempio 3.7. Definiamo induttivamente gli alberi binari in cui i nodi sono numeri $n \in \mathbb{N}$. Ogni numero è un albero binario, perchè è un albero con un solo nodo:

$$n \in BT \quad \text{(Base)}$$

Consideriamo due alberi binari $T_1 \in BT$ e $T_2 \in BT$ allora:

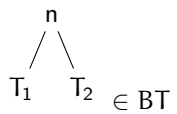

Figura 4: Albero binario composto da due alberi binari

Quindi:

$$br(T_1, T_2) \in BT$$

dove $br()$ è l'operatore che aggiunge un nodo una radice comune a T_1 e T_2

IMPORTANTE: Dimostriamo che la funzione che calcola il numero di foglie (`#foglie(T)`) e la funzione che restituisce il numero di nodi che hanno figli (`#branch(T)`) sono uguali a:

$$\forall T \in BT.\ \texttt{\#foglie}(T) = \texttt{\#branch}(T) + 1$$

Definiamo la funzione `#foglie(T)`:

$$\begin{cases} \#foglie(n) = 1 \\ \#foglie(br(T_1, T_2)) = \#foglie(T_1) + \#foglie(T_2) \end{cases}$$

Definiamo la funzione `#branch(T)`:

$$\begin{cases} \#branch(n) = 0 \\ \#branch(br(T_1, T_2)) = \#branch(T_1) + \#branch(T_2) + 1 \end{cases}$$

Dimostrazione per induzione sulla costruzione degli elementi di BT:

1. **Caso Base**: è il nodo che non ha figli in BT:

$$T = n \in BT$$

Sappiamo che:

$$\#foglie(T) = \#foglie(n) = 1 = 1 + 0 = 1 + \#branch(T)$$

e questo soddisfa la proprietà.

2. **Passo induttivo**: $T = br(T_1, T_2)$. Ipotesi induttiva: supponiamo che T_1 e T_2 soddisfino la proprietà, quindi:

$$\begin{aligned} \#foglie(T_1) &= \#branch(T_1) + 1 \\ \#foglie(T_2) &= \#branch(T_2) + 1 \end{aligned}$$

Dobbiamo dimostrare che $\#foglie(T) = \#branch(T) + 1$:

$$\begin{aligned} \#foglie(T) &\overset{def\ T}{=} \#foglie(br(T_1, T_2)) \\ &\overset{def\ \#foglie}{=} \#foglie(T_1) + \#foglie(T_2) \\ &\overset{Hp\ Ind}{=} \underbrace{\#branch(T_1) + 1 + \#branch(T_2)}_{def\ \#branch(br(T_1,T_2))} + 1 \\ &= \#branch(\underbrace{br(T_1, T_2)}_{T}) + 1 \\ &= \#branch(T) + 1 \end{aligned}$$

Quindi anche T soddisfa la proprietà.

Esempio 3.8. Consideriamo la seguente grammatica:

$$S \to aa \mid bb \mid aSa \mid bSb$$

questa grammatica genera tutte le stringhe palindrome di lunghezza pari. Siccome aa e bb sono nel linguaggio, se si prende una qualsiasi stringa nel linguaggio S e si aggiungono a o b all'inizio e alla fine, si ottiene una stringa ancora palindroma, quindi anche questa stringa è nel linguaggio. Si può quindi definire questa grammatica in modo induttivo:

$$LS : \begin{cases} aa, bb, \in LS & \text{(Base)} \\ \sigma \in LS \Rightarrow a\sigma a, b\sigma b \in LS & \text{(Passo induttivo)} \end{cases}$$

Quindi $L(S) = LS$ e la proprietà da dimostrare è:

$$\sigma \in LS \Rightarrow \exists x, y.\ \sigma = xy \wedge x = reverse(y)$$

1. **Caso base**: Prendiamo:

$$aa = \sigma \Rightarrow x = a, y = a \quad \text{e} \quad x = a = reverse(a) = reverse(y)$$

analogo per $\sigma = bb$.

2. **Passo induttivo**: Prendiamo $\sigma \in LS$, l'ipotesi induttiva è la seguente:

$$\sigma = xy \text{ t.c. } x = reverse(y)$$

Prendiamo $\sigma' = a\sigma a$. Per ipotesi induttiva:

$$\sigma' = a\sigma a = a\overbrace{xy}^{\sigma}a = x'y'$$

con $x' = ax$ e $y' = ya$. Quindi:

$$\begin{aligned} x' = ax &= areverse(y) \\ &= reverse(\underbrace{ya}_{y'}) \\ &= reverse(y') \end{aligned}$$

Abbiamo dimostrato che σ' è palindroma, quindi $\sigma' \in LS$.

Esercizio 3.1. Consideriamo la seguente grammatica:

$$E \to n \mid E + E \mid E * E$$

1. Definire strutturalmente l'insieme $L(E)$
2. Si considerino le seguenti funzioni:
   - $op(E) : op(n) = 0 \quad op(E_1 + E_2) = op(E_1 * E_2) = 1 + op(E_1) + op(e_2)$
   - $val(E) : val(n) = 1 \quad val(E_1 + E_2) = val(E_1 * E_2) = val(E_1) + val(E_2)$

   Dimostrare che:

$$\forall E \,.\, val(E) = op(E) + 1$$